CYBERPUNK DRONE LAB

Interactive 3D Graphics Simulation Environment

Course: Interactive Graphics - Final Project

Sapienza University of Rome - A.Y. 2025/2026

Authors

Zeyad Kandil - Matricola: 2262749

Three.js r128 | WebGL | GLSL Shaders | JavaScript ES6+

June 2026

. Table of Contents

1. Project Overview

The Cyberpunk Drone Lab is an interactive 3D simulation environment built with Three.js (r128) and WebGL. The user pilots a quadrotor drone through a cyberpunk-styled laboratory arena, collecting target rings while avoiding structural columns, all within a 45-second time-attack game mode.

The project demonstrates five core course competencies: hierarchical 3D modelling, lighting and texture application, real-time user interaction, multi-system animation, and comprehensive documentation. The entire application runs in a standard web browser with no plugins beyond WebGL support.

Core Gameplay

- Objective: Fly through as many glowing target rings as possible within 45 seconds.
- Controls: WASD/Arrow keys for horizontal movement, Q/E for vertical ascent/descent.
- Scoring: 1,000 base points per ring, multiplied by combo counter (up to 8x).
- Penalties: Column collisions deduct 500 points and drain battery life.
- Normal difficulty with balanced battery drain and collision penalty values.

Visual Theme

A neon-cyberpunk aesthetic with a dark sci-fi backdrop, emissive materials, additive-blended particle effects, and a custom GLSL energy-shield shader on the drone's visor. The lab environment features a reflective metallic floor with a procedural normal map, illuminated by a hemisphere light, directional shadows, and multiple point-light sources. The drone's visor supports three colour modes (Cyan, Magenta, Gold) with enhanced saturation for vibrant visual feedback.

2. Technical Architecture & Hierarchical Models

The entire 3D world is built using Three.js's scene graph, which provides a natural hierarchical (tree) structure. Every visible object is a node in this tree, with transformations inherited from parent to child. This section describes the major model hierarchies.

2.1 Drone - Hierarchical Assembly

The drone (CyberpunkDrone class in drone.js) is the most complex hierarchical model in the scene. Its construction follows a rooted tree pattern:

```
Scene
  +-- Group: drone.mesh (root)
       +-- Group: chassisGroup
       |    +-- Mesh: Body capsule
       |    +-- Mesh: Visor cone (GLSL shader)
       |    +-- PointLight: Cockpit core
       +-- Group: armAnchor (x4, 90 deg apart)
            +-- Mesh: Arm beam
            +-- Mesh: Motor cap (neon)
            +-- Group: propGroup (spinning)
                 +-- Mesh: Blade
                 +-- Mesh: Center nut
```

- Arm anchors are positioned at (+-1, 0, +-1) and rotated so each beam radiates outward from the centre, creating the classic quadrotor X-frame.
- Propeller groups are children of their respective arm anchors. Applying rotation.y to the prop group spins only the blades while the arm stays stationary, cleanly separating the animated sub-assembly.
- Counter-rotation: Two opposite propellers spin clockwise, the other two counter-clockwise, modelling real quadrotor physics.
- The visor uses a custom ShaderMaterial with GLSL vertex/fragment code from shaders.js, demonstrating custom GPU programs within a hierarchical model.

2.2 Environment - Flat Hierarchy

The lab environment (sceneconfig.js) uses a flatter structure:

- Floor: PlaneGeometry(60, 60) with procedural colour + normal map textures.
- Boundary walls: Four BoxGeometry strips forming an open-top arena.
- Columns: Six CylinderGeometry obstacles with neon band rings, evenly distributed.
- Target rings: Three TorusGeometry objects with emissive materials, animated with rotation and vertical bobbing.

2.3 Hierarchical Modelling - Course Requirement

Requirement met. The drone assembly uses a minimum of three levels of nesting (root - arm anchor - prop group - blade), with local transformations at each level. The structure is modular, reusable, and demonstrates clear parent-child transform inheritance.

3. Lighting and Textures

3.1 Lighting Configuration

The scene uses a multi-source lighting array (setupLightingArray()) with ten individual sources:

Type	Count	Role
HemisphereLight	1	Ambient sky/ground fill (0x4466aa / 0x0a0a1a)
DirectionalLight	2	Primary + fill shadow-casting sun sources
SpotLight	1	Overhead neon spot, target-follows drone
PointLight	6	Deck lights (blue/cyan) at arena corners + centre
PointLight (drone)	1	Cockpit core light, moves with drone

All lights except the hemisphere and directional can be toggled via the 'Toggle Deck Lights' button, giving the user interactive control over the scene's mood lighting.

3.2 Textures

Procedural Colour Map (floor): A 256x256 canvas is drawn with a dark grid pattern: a filled background (0x1a1a2e) overlaid with grid lines (0x2a2a4e) at regular intervals. The canvas is wrapped as a CanvasTexture with RepeatWrapping (16x16 tiles), creating a seamless sci-fi grid floor.

Procedural Normal Map (floor): A second 256x256 canvas generates a bump pattern: a flat tangent-space normal (128, 128, 255) is overlaid with 2,000 random circular bumps of varying sizes and intensities. This creates a rough/metallic surface appearance when used as a normalMap with normalScale (0.4, 0.4). The normal repeats 32x32 across the floor for finer detail.

```
floorMat = new THREE.MeshStandardMaterial({
  map: textureCanvas,          // procedural grid
  normalMap: normalTexture,    // procedural bumps
  normalScale: (0.4, 0.4),
  roughnessMap: textureCanvas,
  roughness: 0.3,
  metalness: 0.9,
  envMapIntensity: 0.6
});
```

3.3 Lighting and Textures - Course Requirement

Requirement met. The scene uses at least three distinct light types (hemisphere, directional, spot, point) and two procedural textures (colour + normal) applied to the floor material.

4. User Interaction

Interaction is handled through an event-driven system registered in setupInteractions(). The interface is divided into flight controls and configuration controls.

4.1 Flight Controls

Input	Action
W / Up Arrow	Move forward (negative Z)
S / Down Arrow	Move backward (positive Z)
A / Left Arrow	Strafe left (negative X)
D / Right Arrow	Strafe right (positive X)
Q	Ascend (positive Y)
E	Descend (negative Y)
SPACE / ENTER	Toggle engine ignition
ESC	Kill engine immediately
C	Cycle camera (Orbit - Follow - Long Shot)
R	Reset simulation

4.2 Configuration Controls

> Drone Colour - Cyan / Magenta / Gold. Updates the visor's GLSL shader uniform uColor and the cockpit point-light colour in real time. Colour saturation has been enhanced for more vibrant visual feedback.

> Throttle Slider - Ranges from 20% to 250%, displayed as a multiplier (x0.2 to x2.5). Controls rotor speed and movement thrust.

> Camera Cycling - Three modes: Orbit (free rotation via OrbitControls), Follow (responsive chase cam with speed-based offset), Long Shot (cinematic aerial orbit).

4.3 Keyboard State Machine

All key presses are tracked in a global activeKeys object via keydown/keyup listeners. The flight loop (processFlightInputs()) reads this object each frame and sets velocity targets proportional to rotor speed. This allows smooth, simultaneous multi-key chording (e.g., W + A + Q for diagonal ascent).

4.4 User Interaction - Course Requirement

Requirement met. The application provides keyboard + mouse + button-panel interaction. Users control flight, cycle cameras, toggle lights, and customise the drone's colour - all in real time with visual feedback.

5. Animation System

The project implements five independent animation systems, each running at the main loop's frame rate (~60 fps).

5.1 Rotor Spin

Each propeller's propGroup rotates around its local Y axis at a rate proportional to rotorSpeed (a smoothed value that lerps toward targetRotorSpeed). Two propellers spin clockwise, two counter-clockwise. Speed ranges from 0 (idle) to ~4,800 RPM at full throttle.

5.2 Ring Animation

Target rings exhibit two simultaneous motions:

- Rotation: Each ring rotates about the Y axis at 0.6 rad/s.
- Vertical bobbing: Each ring oscillates vertically using a sine wave offset by i * 2.1 radians per ring, at amplitude 0.15 m/s and frequency 0.8 Hz.

This creates a gentle, desynchronised floating effect that makes rings feel alive and slightly harder to target.

5.3 Particle Systems

> Ring burst: 20 small coloured spheres burst outward from a collected ring's position, with velocity, gravity, fade, and scale decay over ~1 second.

> Ambient particles: ~100 tiny floating spheres drift through the lab with random velocity and colour cycling between cyan and magenta.

> Score popups: Canvas-based text sprites drift upward and fade, showing scored points and combo multiplier.

5.4 Custom GLSL Shader Animation

The drone's visor uses a ShaderMaterial with a time-based fragment shader (EnergyShieldShader). The uniform uTime increments each frame, driving a pulsing neon energy-shield visual effect. The shader features an animated matrix-style lattice grid with Fresnel edge glow, with colour configurable at runtime between Cyan, Magenta, and Gold.

5.5 Camera Animation

> Follow mode: A smooth lerp-based chase camera that responds to drone speed (tighter follow at higher speeds).

> Long Shot mode: A slow cinematic orbit around the drone's position at radius 25m, height 15m.

5.6 Animations - Course Requirement

Requirement met. Multiple independent animation systems run concurrently: mechanical (rotors), environmental (rings), particle effects (burst, ambient, popups), GPU-based (shader), and camera motion (follow + orbit). Animations are frame-rate independent (driven by dt deltas) and use hierarchical transforms, canvas drawing, and custom shaders.

6. Scene Configuration & Obstacle Course

6.1 Environment Layout

> Arena: 60x60 units, bounded by four 1-unit-thick walls, open top.
> Floor: Procedural grid texture with normal map for metallic appearance.
> Columns: Six 3-unit-radius pillars at evenly spaced positions, each wrapped with a neon band ring at mid-height. Used as obstacles with collision detection.
> Target rings: Three torus rings (radius 1.5, tube 0.15) with emissive cyan material. Respawning - when collected, a ring teleports to a new random position.

6.2 Collision System

The function checkCollisions() in drone.js performs per-frame distance checks:

- Column collision: If the drone's XZ distance to any column centre is less than col.radius + 1.0 and the drone is within +-4 units of the column's Y position, the drone is pushed out and a penalty (500 points) is applied.

- Ring scoring: If the drone's XZ distance to a ring is less than ring.radius and vertical distance is less than 0.8 units, the ring is scored, triggering a burst particle effect, score popup, combo tracking, and ring respawn.

6.3 Game State Machine

> Idle - Drone on ground, engine off. Timer not running.
> Flying - Engine on, 45-second countdown active, scoring enabled.
> Game Over - Timer reaches 0, engine killed, final score displayed with battery bonus. High score persisted to localStorage.

7. Requirements Compliance Matrix

The following table maps each course requirement to its implementation:

#	Requirement	Status	Implementation
1	Hierarchical 3D Models (min. 3 levels)	PASS	Drone: root - arm anchor - prop group - blade (4 levels). All transforms local.
2	Lighting & Textures (min. 3 light types, procedural)	PASS	10 lights (hemisphere, directionalx2, spot, pointx6). Procedural colour + normal maps.
3	User Interaction (keyboard, mouse, UI)	PASS	WASD flight, camera cycle, colour config, throttle, light toggle, reset.
4	Animations (mechanical, environmental, GPU)	PASS	Rotor spin, ring bob/rotate, particles, GLSL shader, camera motion.
5	Report (5-10 page document)	PASS	This document covering all technical aspects with code references.

8. Development Environment & Dependencies

8.1 Runtime

Component	Version / Detail
Browser	Any modern browser with WebGL (Chrome 90+, Firefox 88+, Edge 90+)
Three.js	r128 (CDN)
OrbitControls	r128/examples/js/controls/OrbitControls.js
Tween.js	18.6.4 (CDN)
WebGL	1.0 / 2.0 with antialiasing and PCF soft shadows

8.2 File Structure

```
IG Final Project/
  +-- index.html          # Entry point, HUD layout, CSS styling
  +-- app.js              # Core engine, input, game loop, scoring
  +-- sceneconfig.js      # Environment, lights, obstacles, animations
  +-- drone.js            # CyberpunkDrone class (hierarchical model)
  +-- shaders.js          # GLSL vertex/fragment shaders
  +-- report.pdf          # Project report
  +-- README.md           # Documentation
```

8.3 No External Build Tools

The project uses zero build tools, bundlers, or transpilers. All dependencies are loaded via CDN script tags. The application is opened directly in a browser via file:// protocol or any HTTP server. This was a deliberate design constraint to ensure maximum portability.

9. Conclusion

The Cyberpunk Drone Lab demonstrates a comprehensive interactive 3D graphics application built entirely with web standards. The project satisfies all five course requirements:

1. Hierarchical 3D models via a quadrotor drone with a four-level nested transform hierarchy.
2. Lighting and textures through a ten-source lighting array and two procedurally generated texture maps (colour + normal).
3. User interaction with keyboard flight controls, camera modes, colour customisation, and a fully featured HUD.
4. Animations spanning rotor mechanics, environmental ring motion, particle effects, GPU shader programs, and camera choreography.
5. Report delivered as this document with technical analysis, code references, and a requirements compliance matrix.

The game loop runs at a stable 60 fps on modern hardware, with frame-rate independent physics and animation. The scoring system with combos, penalties, and persistent high scores creates engaging replayability. The modular .js architecture separates concerns cleanly (engine, scene, drone, shaders), making future extension straightforward.

Enhancements - procedural normal map, ring animations, drone colour configuration with enhanced saturation - bring the project to full compliance with all course specifications.

- End of Report -